

Collaborative Text Editing with Eg-walker: Better, Faster, Smaller

Joseph Gentle
me@josephg.com
Independent
Melbourne, Australia

Martin Kleppmann
martin.kleppmann@cst.cam.ac.uk
University of Cambridge
Cambridge, United Kingdom

Abstract

Collaborative text editing algorithms allow several users to concurrently modify a text file, and automatically merge concurrent edits into a consistent state. Existing algorithms fall in two categories: Operational Transformation (OT) algorithms are slow to merge files that have diverged substantially due to offline editing; CRDTs are slow to load and consume a lot of memory. We introduce Eg-walker, a collaboration algorithm for text that avoids these weaknesses. Compared to existing CRDTs, it consumes an order of magnitude less memory in the steady state, and loading a document from disk is orders of magnitude faster. Compared to OT, merging long-running branches is orders of magnitude faster. In the worst case, the merging performance of Eg-walker is comparable with existing CRDT algorithms. Eg-walker can be used everywhere CRDTs are used, including peer-to-peer systems without a central server. By offering performance that is competitive with centralised algorithms, our result paves the way towards the widespread adoption of peer-to-peer collaboration software.

CCS Concepts: • **Applied computing** → **Text editing**; • **Human-centered computing** → **Computer supported cooperative work**; • **Information systems** → *Asynchronous editors*; • **Computing methodologies** → *Distributed algorithms*.

Keywords: collaborative text editing, CRDTs, operational transformation, strong eventual consistency

ACM Reference Format:

Joseph Gentle and Martin Kleppmann. 2025. Collaborative Text Editing with Eg-walker: Better, Faster, Smaller. In *Twentieth European Conference on Computer Systems (EuroSys '25)*, March 30-April 3, 2025, Rotterdam, Netherlands. ACM, New York, NY, USA, 18 pages. <https://doi.org/10.1145/3689031.3696076>



This work is licensed under a Creative Commons Attribution 4.0 International License.

EuroSys '25, March 30-April 3, 2025, Rotterdam, Netherlands

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1196-1/25/03

<https://doi.org/10.1145/3689031.3696076>

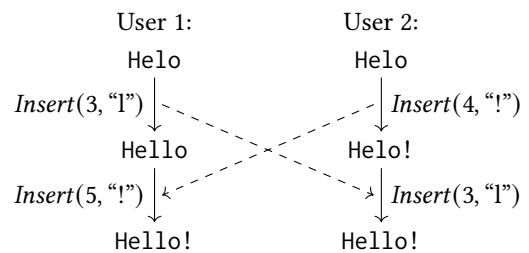


Figure 1. Two concurrent insertions into a text document.

1 Introduction

Real-time collaboration has become an essential feature for many types of software, including document editors such as Google Docs, Microsoft Word, or Overleaf, and graphics software such as Figma. In such software, each user's device locally maintains a copy of the shared file (e.g. in a tab of their web browser). A user's edits are immediately applied to their own local copy, without waiting for a network round-trip, so that the user interface is responsive regardless of network latency. Different users may therefore make edits concurrently; the software must merge such concurrent edits in a way that maintains the integrity of the document, and ensures that all devices converge to the same state.

For example, in Figure 1, two users initially have the same document "Helo". User 1 inserts a second letter "l" at index 3, while concurrently user 2 inserts an exclamation mark at index 4. When user 2 receives the operation *Insert*(3, "l") they can apply it to obtain "Hello!", but when user 1 receives *Insert*(4, "!") they cannot apply that operation as-is, since that would result in the state "Hell!o", which would be inconsistent with the other user's state and the intended insertion position. Due to the concurrent insertion at an earlier index, user 1 must insert the exclamation mark at index 5.

One way of solving this problem is to use *Operational Transformation* (OT): when user 1 receives *Insert*(4, "!") that operation is transformed with regard to the concurrent insertion at index 3, which increments the index at which the exclamation mark is inserted. OT is an old and widely-used technique: it was introduced in 1989 [18], and the OT algorithm Jupiter [44] is used in Google Docs [17].

OT is simple and fast in the case of Figure 1, where each user performed only one operation since the last version they had in common. In general, if the users each performed

n operations since their last common version, merging their states using OT has a cost of at least $O(n^2)$, since each of one user's operations must be transformed with respect to all of the other user's operations. Some OT algorithms' merge complexity is cubic or even slower [39, 52, 57]. This is acceptable for online collaboration where n is typically small, but for larger n an algorithm with complexity $O(n^2)$ can become impractically slow. In Section 4 we show a real-life example document that takes one hour to merge using OT.

Larger divergence occurs if users may edit a document offline, or if the software supports explicit branching and merging workflows. In version control systems like Git, used mostly for software development, offline working and explicit branching are already the norm. Recent research indicates that such workflows would also be valuable for writing prose [40, 42], but OT-based collaborative editors struggle to offer such features because of the cost of merging substantially diverged branches.

Conflict-free Replicated Data Types (CRDTs) have been proposed as an alternative to OT. The first CRDT for collaborative text editing appeared in 2006 [47], and over a dozen text CRDTs have been published since [35]. These algorithms work by maintaining additional metadata: they give each character a unique identifier, and use those IDs instead of integer indexes to identify the position of insertions and deletions. This avoids having to transform operations, since IDs are not affected by concurrent operations.

Unfortunately, these IDs need to be loaded from disk when a document is opened, and held in memory while a document is being edited. Some CRDT algorithms also need to retain IDs of deleted characters (*tombstones*). Early CRDT algorithms were very inefficient, using hundreds of bytes of memory for each character of text, making them impractical for long documents. Recent CRDT implementations have reduced this overhead considerably, but as we show in Section 4, even the best CRDTs available today use more than 10 times as much memory as OT to view and edit a document. For this reason, popular apps like Google Docs [17], Microsoft Office, and Overleaf [48] use OT. Existing algorithms therefore present a trade-off: either use OT and accept that offline editing and long-running branches are slow, or pick a CRDT and accept a much higher memory use.

In this paper we propose *Event Graph Walker* (Eg-walker), a collaborative editing algorithm that overcomes this trade-off. Like OT, Eg-walker uses integer indexes to identify insertion and deletion positions, and transforms those indexes to merge concurrent operations. When two users concurrently perform n operations each, Eg-walker can merge them at a cost of $O(n \log n)$, much faster than OT's cost of $O(n^2)$ or worse. The example document that takes 1 hour to merge using OT is merged in just 24 ms using Eg-walker (Figure 8).

Eg-walker merges concurrent edits using a CRDT algorithm we designed. Unlike existing algorithms, we invoke the CRDT only to perform merges of concurrent operations,

and we discard its state as soon as the merge is complete. We never write the CRDT state to disk and never send it over the network. While a document is being edited, we only hold the document text in memory, but no CRDT metadata. Most of the time, Eg-walker therefore uses 1–2 orders of magnitude less memory than the best CRDTs. During merging, when Eg-walker temporarily uses more memory, its peak memory use is comparable to the best known CRDT implementations.

Eg-walker assumes no central server, so it can be used over a peer-to-peer network. Although all existing CRDTs and a few OT algorithms can be used peer-to-peer, most of them have poor performance compared to the centralised OT commonly used in production software. In contrast, Eg-walker's performance matches or surpasses that of centralised algorithms. It therefore paves the way towards more collaboration software working peer-to-peer, for example in environments where co-located devices can communicate via local radio links, but not reach the Internet or any cloud services. This setting is important e.g. for devices onboard the same aircraft [50], in a military context [19], or for scientists conducting fieldwork in remote locations [15].

This paper focuses on collaborative editing of plain text files. We believe that our approach can be generalised to other file types such as rich text, spreadsheets, graphics, presentations, CAD drawings, and more in the future. More generally, Eg-walker provides a framework for efficient coordination-free distributed systems, in which nodes can always make progress independently, but converge eventually [29].

This paper makes the following contributions:

- We introduce Eg-walker, a hybrid CRDT/OT algorithm for text that is faster and has a vastly smaller memory footprint than existing CRDTs (Section 3).
- Since there is no established benchmark for collaborative text editing, we are also publishing a suite of editing traces of text files for benchmarking. They are derived from real documents and demonstrate various patterns of sequential and concurrent editing.
- In Section 4 we use those editing traces to evaluate the performance of our implementation of Eg-walker, comparing it to selected CRDTs and an OT implementation. We measure CPU time to load a document, CPU time to merge edits from a remote replica, memory usage, and file size. Eg-walker improves the state of the art by orders of magnitude in the best cases, and is only slightly slower in the worst cases.
- We prove the correctness of Eg-walker in the extended version of this paper [26].

2 Background

We consider a collaborative plain text editor whose state is a linear sequence of characters, which may be edited by inserting or deleting characters at any position. Such an edit is captured as an *operation*; the operation *Insert*(i, c)

inserts character c at index i , and $Delete(i)$ deletes the character at index i (indexes are zero-based). Our implementation compresses runs of consecutive insertions or deletions, but for simplicity we describe the algorithm in terms of single-character operations.

2.1 System model

Each device on which a user edits a document is a *replica*, and each replica stores the full editing history of the document. When a user makes an insertion or deletion, that operation is immediately applied to the user's local replica, and then asynchronously sent over the network to any other replicas that have a copy of the same document. Users can also edit their local copy while offline; the corresponding operations are then enqueued and sent when the device is next online.

Our algorithm assumes a reliable broadcast protocol that detects and retransmits lost messages, but makes no other assumptions about the network. For example, a relay server could store and forward messages from one replica to the others, or replicas could use a peer-to-peer gossip protocol. We make no timing assumptions and tolerate arbitrary network delay, but we assume replicas are non-Byzantine.

Our algorithm ensures *convergence*: any two replicas that have seen the same operations have the same document state (i.e., a text consisting of the same sequence of characters), even if the operations arrived in a different order at each replica. If the underlying broadcast protocol ensures that every non-crashed replica eventually receives every operation, the algorithm achieves *strong eventual consistency* [53].

2.2 Event graphs

We represent the editing history of a document as an *event graph*: a directed acyclic graph (DAG) in which every node is an *event* consisting of an operation (insert/delete a character), a unique ID, and the set of IDs of its *parent events*. When a is a *parent* of b , we also say b is a *child* of a , and the graph contains an edge from a to b . We construct events such that the graph is transitively reduced (i.e., it contains no redundant edges). When there is a directed path from a to b we say that a *happened before* b , and write $a \rightarrow b$ as per Lamport [38]. The $\rightarrow$ relation is a strict partial order. We say that events a and b are *concurrent*, written $a \parallel b$, if both events are in the graph, $a \neq b$, and neither happened before the other: $a \not\rightarrow b \wedge b \not\rightarrow a$.

The *frontier* is the set of events with no children. Whenever a user performs an operation, a new event containing that operation is added to the graph, and the previous frontier in the replica's local copy of the graph becomes the new event's parents. The new event is then broadcast over the network, and each replica adds it to its copy of the graph. If any parents are missing (i.e., a parent ID in the event does not resolve to a known event), the replica waits for them to arrive before adding them to the graph; the result is a simple causal broadcast protocol [11, 13]. Two replicas can merge

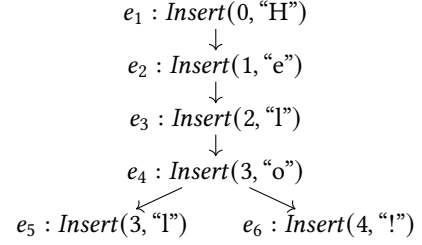


Figure 2. The event graph corresponding to Figure 1.

their event graphs by taking the union of their sets of events. Events in the graph are immutable; they always represent the operation as originally generated, and not as a result of any transformation. The graph grows monotonically (we never remove events), and a new event is always a child of existing events (we never add a parent to an existing event).

For example, Figure 2 shows the event graph corresponding to Figure 1. The events e_5 and e_6 are concurrent, and the frontier of this graph is the set of events $\{e_5, e_6\}$.

The event graph for a substantial document, such as a research paper, may contain hundreds of thousands of events. It can nevertheless be stored in a very compact form by exploiting the typical editing patterns of humans writing text: characters tend to be inserted or deleted in consecutive runs. Many portions of a typical event graph are linear, with each event having one parent and one child. We describe the storage format in more detail in Section 3.8.

2.3 Document versions

Let G be an event graph, represented as a set of events. Due to convergence, any two replicas that have the same set of events must be in the same state. Therefore, the document state (sequence of characters) resulting from G must be $\text{replay}(G)$, where replay is some pure (deterministic and non-mutating) function. In principle, any pure function of the set of events results in convergence, although a replay function that is useful for text editing must satisfy additional criteria (see Section 3.1).

Consider the event $Delete(i)$, which deletes the character at position i in the document. In order to correctly interpret this event, we need to determine which character was at index i at the time when the operation was generated.

More generally, let e_i be some event. The document state when e_i was generated must be $\text{replay}(G_i)$, where G_i is the set of events that were known to the generating replica at the time when e_i was generated (not including e_i itself). By definition, the parents of e_i are the frontier of G_i , and thus G_i is the set of all events that happened before e_i , i.e., e_i 's parents and all of their ancestors. Therefore, the parents of e_i unambiguously define the document state in which e_i must be interpreted.

To formalise this, given an event graph (set of events) G , we define the *version* of G to be its frontier set:

$$\text{Version}(G) = \{e_1 \in G \mid \nexists e_2 \in G : e_1 \rightarrow e_2\}$$

Given some version V , the corresponding set of events can be reconstructed as follows:

$$\text{Events}(V) = V \cup \{e_1 \mid \exists e_2 \in V : e_1 \rightarrow e_2\}$$

Since an event graph grows only by adding events that are concurrent to or children of existing events (we never change the parents of an existing event), there is a one-to-one correspondence between an event graph and its version. For all valid event graphs G , $\text{Events}(\text{Version}(G)) = G$.

The set of parents of an event in the graph is the version of the document in which that operation must be interpreted. The version can hence be seen as a *logical clock*, describing the point in time at which a replica knows about the exact set of events in G . Even if the event graph is large and there are many collaborators, a version rarely consists of more than two events in practice: a version with n events occurs only if n mutually concurrent events are merged with no new operations being generated in the intervening time.

2.4 Replaying editing history

Collaborative editing algorithms are usually defined in terms of sending and receiving messages over a network. The abstraction of an event graph allows us to reframe these algorithms in a simpler way: a collaborative text editing algorithm is a pure function $\text{replay}(G)$ of an event graph G . This function can use the parent-child relationships to partially order events, but concurrent events could be processed in any order. This allows us to separate the process of replicating the event graph from the algorithm that ensures convergence. In fact, this is how *pure operation-based CRDTs* [9] are formulated, as discussed in Section 5.

In addition to determining the document state from an entire event graph, we need an *incremental update* function. Say we have an existing event graph G and corresponding document state $\text{doc} = \text{replay}(G)$. Then an event e from a remote replica is added to the graph. We could rerun the function to obtain $\text{doc}' = \text{replay}(G \cup \{e\})$, but it would be inefficient to process the entire graph again. Instead, we need to efficiently compute the operation to apply to doc in order to obtain doc' . For text documents, this incremental update is also described as an insertion or deletion at a particular index; however, the index may differ from that in the original event due to the effects of concurrent operations, and a deletion may turn into a no-op if the same character has also been deleted by a concurrent operation.

Both OT and CRDT algorithms focus on this incremental update. If none of the events in G are concurrent with e , OT is straightforward: the incremental update is identical to the operation in e , as no transformation takes place. If there is

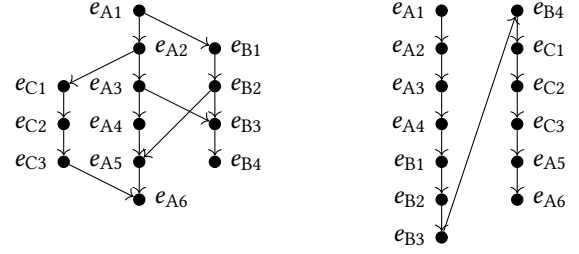


Figure 3. An event graph (left) and one possible topologically sorted order of that graph (right).

concurrency, OT must transform each new event with regard to each existing event that is concurrent to it.

In CRDTs, each event is first translated into operations that use unique IDs instead of indexes, and then these operations are applied to a data structure that reflects all of the operations seen so far (both concurrent operations and those that happened before). In order to update the text editor, these updates to the CRDT's internal structure need to be translated back into index-based insertions and deletions. Many CRDT papers elide this translation from unique IDs back to indexes, but it is important for practical applications.

Regardless of whether the OT or the CRDT approach is used, a collaborative editing algorithm can be boiled down to an incremental update to an event graph: given an event to be added to an existing event graph, return the (index-based) operation that must be applied to the current document state so that the resulting document is identical to replaying the entire event graph including the new event.

2.5 Implementing OT using a CRDT

One way of implementing such a replay algorithm would be to simulate a network of CRDT replicas in a single process. For each branch in the event graph there is a separate simulated replica, which takes operations in their original index-based form and generates a corresponding ID-based CRDT operation. Another simulated replica receives every operation generated by the other replicas and applies them in some topologically sorted order, as illustrated in Figure 3.

For example, the history in Figure 3 could be replayed using one simulated replica for $e_{A1} \dots e_{A6}$, a second for $e_{B1} \dots e_{B4}$, and a third for $e_{C1} \dots e_{C3}$. Every time an event's parent is an event generated on another simulated replica, the corresponding network communication is simulated, and the remote operations are merged using a CRDT algorithm. For example, before the replica for $e_{B1} \dots e_{B4}$ can generate e_{B3} it must first merge e_{A2} and e_{A3} . Each simulated replica thus tracks the document version in which the indexes of insertions and deletions should be interpreted. The simulated replica that applies all operations then converts the ID-based operation

back into an index based on its document version. This index-based operation then allows an incremental update of the document state.

The process of translating an index-based operation into an ID-based one on one simulated replica, and translating it back into an index-based operation on another, is effectively an operational transformation algorithm: it updates the index to reflect the effects of concurrent operations (which have been applied to the second simulated replica but not the first). However, the algorithm is fairly slow because it incurs the overhead of updating multiple simulated replicas and running the CRDT algorithm even at times when there is no concurrency in the event graph. It also uses a lot of memory because it needs a separate copy of the CRDT state for every concurrent branch in the event graph.

Our Eg-walker algorithm, described in the next section, modifies this approach to use only two simulated replicas: one on which operations are generated, and the other on which all operations are applied (and in fact, both are stored in the same data structure). To deal with event graphs that are not totally ordered, the algorithm allows events on one branch to be *retreated* when switching to another branch, and *advanced* again when those branches are merged. Retreating an event updates the replica state to behave as if that event had not yet happened, and advancing makes the event take effect again.

For example, in Figure 3, after applying e_{A4} we would retreat e_{A4} , e_{A3} , and e_{A2} before applying e_{B1} , since those events are concurrent with e_{B1} . Before applying e_{B3} we would advance e_{A2} and e_{A3} again, since they are ancestors of e_{B3} . Retreating and advancing takes some additional CPU time on highly concurrent event graphs, but as we show in Section 4, the optimisations this approach enables result in excellent performance overall.

3 The Event Graph Walker algorithm

Eg-walker is a collaborative text editing algorithm based on the idea of event graph replay. The algorithm builds on a replication layer that ensures that whenever a replica adds an event to the graph, all non-crashed replicas eventually receive it. The state of each replica consists of three parts:

1. **Event graph:** Each replica stores a copy of the event graph on disk, in a format described in Section 3.8.
2. **Document state:** The current sequence of characters in the document with no further metadata. On disk this is simply a plain text file; in memory it may be represented as a rope [12], piece table [41], or similar structure to support efficient insertions and deletions.
3. **Internal state:** A temporary CRDT structure that Eg-walker uses to merge concurrent edits. It is not persisted or replicated, and it is discarded when the algorithm finishes running.

Eg-walker can reconstruct the document state by replaying the entire event graph. It first performs a topological sort, as illustrated in Figure 3. Then each event is transformed so that the transformed insertions and deletions can be applied in topologically sorted order, starting with an empty document, to obtain the document state. In Git parlance, this process “rebases” a DAG of operations into a linear operation history with the same effect. The input of the algorithm is the event graph, and the output is this topologically sorted sequence of transformed operations. While OT transforms one operation with respect to one other, Eg-walker uses the internal state to transform sets of operations efficiently.

In graphs with concurrent operations there are multiple possible sort orders. Eg-walker guarantees that the final document state is the same, regardless which of these orders is chosen. However, the choice of sort order may affect the performance of the algorithm, as discussed in Section 3.7.

For example, the graph in Figure 2 has two possible sort orders; Eg-walker either first inserts “l” at index 3 and then “!” at index 5 (like User 1 in Figure 1), or it first inserts “!” at index 4 followed by “l” at index 3 (like User 2 in Figure 1). The final document state is “Hello!” either way.

Event graph replay easily extends to incremental updates for real-time collaboration: when a new event is added to the graph, it becomes the next element of the topologically sorted sequence. We can transform each new event in the same way as during replay, and apply the transformed operation to the current document state.

3.1 Characteristics of Eg-walker

Eg-walker ensures that the resulting document state is consistent with Attiya et al.’s *strong list specification* [8] (in essence, replicas converge to the same state and apply operations in the right place), and it is *maximally non-interleaving* [60] (i.e., concurrent sequences of insertions at the same position are placed one after another, and not interleaved).

When generating new events, or when adding an event to the graph that happened after all existing events, Eg-walker only needs the current document state. Most of the time, the event graph can thus remain on disk without using any space in memory or any CPU time, and the internal state can be discarded entirely. The event graph and internal state are only required when handling concurrency, and even then we only have to replay the portion of the graph since the last ancestor that the concurrent operations had in common. In portions of the event graph that have no concurrency (which, in many editing histories, is the vast majority of events), events do not need to be transformed at all.

In contrast, existing CRDTs require every replica to persist the internal state and send it over the network. They also require that state to be loaded into memory to generate and receive operations, even when there is no concurrency. This uses several times more memory and makes documents slow to load.

OT algorithms avoid this internal state; similarly to Eg-walker, they only need to persist the latest document state and the history of operations that are concurrent to operations that may arrive in the future. In both Eg-walker and OT, the event graph can be discarded if we know that no event we may receive in the future will be concurrent with any existing event. However, OT algorithms are very slow to merge long-running branches (see Section 4). Eg-walker handles arbitrary event DAGs, whereas some OT algorithms are only able to handle restricted forms of event graphs (server-based OT corresponds to event graphs with one main branch representing the server’s view; all other branches may merge to and from the main branch, but not with each other).

3.2 Walking the event graph

For the sake of clarity we first explain a simplified version of Eg-walker that replays the entire event graph without discarding its internal state along the way. This approach incurs some CRDT overhead even for non-concurrent operations. We give pseudocode for this simplified algorithm in the extended version of this paper [26]. In Section 3.6 we show how the algorithm can be optimised to replay only a part of the event graph.

First, we topologically sort the event graph in a way that keeps events on the same branch consecutive as much as possible: for example, in Figure 3 we first visit $e_{A1} \dots e_{A4}$, then $e_{B1} \dots e_{B4}$. We avoid alternating between branches, such as $e_{A1}, e_{B1}, e_{A2}, e_{B2} \dots$, even though that would also be a valid topological sort. For this we use a standard textbook algorithm [16]: perform a depth-first traversal starting from the oldest event, and build up the topologically sorted list in the order that events are visited. When a node has multiple children in the graph, we choose their order based on a heuristic so that branches with fewer events tend to appear before branches with more events in the sorted order; this can improve performance (see Section 3.7) but is not essential. We estimate the size of a branch by counting the number of events that happened after each event.

The algorithm then processes the events one at a time in topologically sorted order, updating the internal state and outputting a transformed operation for each event. The internal state simultaneously captures the document at two versions: the version in which an event was generated (which we call the *prepare* version), and the version in which all events seen so far have been applied (which we call the *effect* version). These correspond to the two simulated replicas mentioned in Section 2.5. If the prepare and effect versions are the same, the transformed operation is identical to the original one. In general, the prepare version represents a subset of the events of the effect version.

The internal state can be updated with three methods, each of which takes an event as argument:

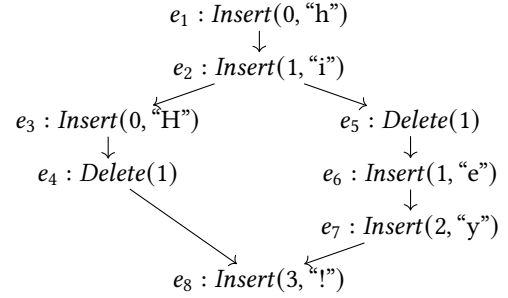


Figure 4. An event graph. Starting with document “hi”, one user changes “hi” to “hey”, while concurrently another user capitalises the “H”. After merging to the state “Hey”, one of them appends an exclamation mark to produce “Hey!”.

- $\text{apply}(e)$ updates the prepare version and the effect version to include e , assuming that the current prepare version equals $e.\text{parents}$, and that e has not yet been applied. This method interprets e in the context of the prepare version, and outputs the operation representing how the effect version has been updated.
- $\text{retreat}(e)$ updates the prepare version to remove e , assuming the prepare version previously included e .
- $\text{advance}(e)$ updates the prepare version to add e , assuming that the prepare version previously did not include e , but the effect version did.

The effect version only moves forwards in time (through apply), whereas the prepare version can move both forwards and backwards. Consider the example in Figure 4, and assume that the events $e_1 \dots e_8$ are traversed in order of their subscript. These events can be processed as follows:

1. Start in the empty state, then call $\text{apply}(e_1)$, $\text{apply}(e_2)$, $\text{apply}(e_3)$, and $\text{apply}(e_4)$. This is valid because each event’s parent version is the previously applied event.
2. Before we can apply e_5 we must rewind the prepare version to be $\{e_2\}$, which is the parent of e_5 . We can do this by calling $\text{retreat}(e_4)$ and $\text{retreat}(e_3)$.
3. Now we can call $\text{apply}(e_5)$, $\text{apply}(e_6)$, and $\text{apply}(e_7)$.
4. The parents of e_8 are $\{e_4, e_7\}$; before we can apply e_8 we must therefore add e_3 and e_4 to the prepare state again by calling $\text{advance}(e_3)$ and $\text{advance}(e_4)$. We do not retreat $e_{5..7}$ because e_3 and e_4 have already been applied in Step 1; now we are *advancing* e_3 and e_4 , which does not require retreating concurrent events.
5. Finally, we can call $\text{apply}(e_8)$.

In complex event graphs such as the one in Figure 3 the same event may have to be retreated and advanced several times, but we can process arbitrary DAGs this way. In general, before applying the next event e in topologically sorted order, compute $G_{\text{old}} = \text{Events}(V_p)$ where V_p is the current prepare version, and $G_{\text{new}} = \text{Events}(e.\text{parents})$. We then call retreat on each event in $G_{\text{old}} - G_{\text{new}}$ (in reverse topological sort

order), and call *advance* on each event in $G_{\text{new}} - G_{\text{old}}$ (in topological sort order) before calling *apply*(e).

The following algorithm efficiently computes the events to retreat and advance when moving the prepare version from V_p to V'_p . For each event in V_p and V'_p we insert the index of that event in the topological sort order into a priority queue, along with a tag indicating whether the event is in the old or the new prepare version. We then repeatedly pop the event with the greatest index off the priority queue, and enqueue the indexes of its parents along with the same tag. We stop the traversal when all entries in the priority queue are common ancestors of both V_p and V'_p . Any events that were traversed from only one of the versions need to be retreated or advanced respectively.

3.3 Representing prepare and effect versions

The internal state implements the *apply*, *retreat*, and *advance* methods by maintaining a CRDT data structure. This structure consists of a linear sequence of records, one per character in the document, including tombstones for deleted characters. Runs of characters with consecutive IDs and the same properties can be run-length encoded to save memory. A record is inserted into this sequence by *apply*(e_i) for an insertion event e_i . Subsequent deletion events and *retreat*/*advance* calls may modify properties of the record, but records in the sequence are not removed or reordered once they have been inserted.

When the event graph contains concurrent insertions, we use a CRDT to ensure that all replicas place the records in this sequence in the same order, regardless of the order in which the event graph is traversed. For example, RGA [52] or YATA [45] could be used for this purpose. Our implementation of Eg-walker uses a variant of the Yjs algorithm [30], itself based on YATA, that we conjecture to be maximally non-interleaving. We leave a detailed analysis of this algorithm to future work, since it is not core to this paper.

Each record in this sequence contains:

- the ID of the event that inserted the character;
- $s_p \in \{\text{NotInsertedYet}, \text{Ins}, \text{Del } 1, \text{Del } 2, \dots\}$, the character's state in the prepare version;
- $s_e \in \{\text{Ins}, \text{Del}\}$, the state in the effect version;
- and any other fields required by the CRDT to determine the order of concurrent insertions.

The rules for updating s_p and s_e are:

- When a record is first inserted by *apply*(e_i) with an insertion event e_i , it is initialised with $s_p = s_e = \text{Ins}$.
- If *apply*(e_d) is called with a deletion event e_d , we set $s_e = \text{Del}$ in the record representing the deleted character. In the same record, if $s_p = \text{Ins}$ we update it to $\text{Del } 1$, and if $s_p = \text{Del } n$ it advances to $\text{Del}(n + 1)$, as shown in Figure 5.
- If *retreat*(e_i) is called with insertion event e_i , we must have $s_p = \text{Ins}$ in the record affected by the event, and

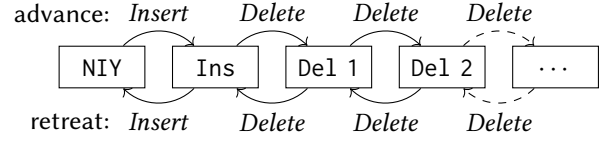


Figure 5. State machine for internal state variable s_p .

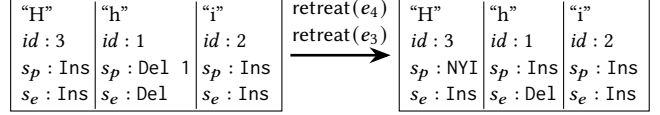


Figure 6. Left: the internal state after applying $e_1 \dots e_4$ from Figure 4. Right: after *retreat*(e_4) and *retreat*(e_3), the prepare state is updated to mark “H” as NotInsertedYet, and the deletion of “h” is undone. The effect state is unchanged.

“H”	“h”	“e”	“y”	“!”	“!”
<i>id</i> : 3	<i>id</i> : 1	<i>id</i> : 6	<i>id</i> : 7	<i>id</i> : 8	<i>id</i> : 2
s_p : Ins	s_p : Del 1	s_p : Ins	s_p : Ins	s_p : Ins	s_p : Del 1
s_e : Ins	s_e : Del	s_e : Ins	s_e : Ins	s_e : Ins	s_e : Del

Figure 7. The internal Eg-walker state after replaying all of the events in Figure 4.

we update it to $s_p = \text{NotInsertedYet}$. Conversely, *advance*(e_i) moves s_p from NotInsertedYet to Ins.

- If *retreat*(e_d) is called with a deletion event e_d , we must have $s_p = \text{Del } n$ in the affected record, and we update it to $\text{Del}(n - 1)$ if $n > 1$, or to Ins if $n = 1$. Calling *advance*(e_d) performs the opposite.

As a result, s_p and s_e are Ins if the character is visible (inserted but not deleted) in the prepare and effect version respectively; $s_p = \text{Del } n$ indicates that the character has been deleted by n concurrent delete events in the prepare version; and $s_p = \text{NotInsertedYet}$ indicates that the insertion of the character has been retreated in the prepare version. s_e does not count the number of deletions and does not have a NotInsertedYet state since we never remove the effect of an operation from the effect version.

For example, Figure 6 shows the internal state after applying $e_1 \dots e_4$ from Figure 4, and how that state is updated by retreating e_4 and e_3 before e_5 is applied. In the effect state, the lowercase “h” is marked as deleted, while the uppercase “H” and the “i” are visible. In the prepare state, by retreating e_4 and e_3 the “H” is marked as NotInsertedYet, and the deletion of “h” is undone ($s_p = \text{Ins}$).

Figure 7 shows the state after replaying all of the events in Figure 4: “i” is also deleted, the characters “e” and “y” are inserted immediately after the “h”, e_3 and e_4 are advanced again, and finally “!” is inserted after the “y”. The figures include the character for the sake of readability, but Eg-walker actually does not store text content in its internal state.

3.4 Mapping indexes to character IDs

In the event graph, insertion and deletion operations specify the index at which they apply. In order to update Eg-walker's internal state, we need to map these indexes to the correct record in the sequence, based on the prepare state s_p . To produce the transformed operations, we need to map the positions of these internal records back to indexes again – this time based on the effect state s_e .

A simple but inefficient algorithm would be: to apply a *Delete*(i) operation we iterate over the sequence of records and pick the i th record with a prepare state of $s_p = \text{Ins}$ (i.e., the i th among the characters that are visible in the prepare state, which is the document state in which the operation should be interpreted). Similarly, to apply *Insert*(i, c) we skip over $i - 1$ records with $s_p = \text{Ins}$ and insert the new record after the last skipped record (if there have been concurrent insertions at the same position, we may also need to skip over some records with $s_p = \text{NotInsertedYet}$, as determined by the list CRDT's insertion ordering).

To reduce the cost of this algorithm from $O(n)$ to $O(\log n)$, where n is the number of characters in the document, we construct a B-tree whose leaves, from left to right, contain the sequence of records representing characters. We extend the tree into an *order statistic tree* [16] (also known as *ranked B-tree*) by adding two integers to each node: the number of records with $s_p = \text{Ins}$ contained within that subtree, and the number of records with $s_e = \text{Ins}$ in that subtree. Every time s_p or s_e are updated, we also update those numbers on the path from the updated record to the root. As the tree is balanced, this update takes $O(\log n)$.

Now we can find the i th record with $s_p = \text{Ins}$ in logarithmic time by starting at the root of the tree, and adding up the values in the subtrees that have been skipped. Moreover, once we have a record in the sequence we can efficiently determine its index in the effect state by going in the opposite direction: working upwards in the tree towards the root, and summing the numbers of records with $s_e = \text{Ins}$ that lie in subtrees to the left of the starting record. This allows us to efficiently transform the index of an operation from the prepare version into the effect version. If the character was already deleted in the effect version ($s_e = \text{Del}$), the transformed operation is a no-op.

The above process makes *apply*(e_i) efficient. We also need to efficiently perform *retreat*(e_i) and *advance*(e_i), which modify the prepare state s_p of the record inserted or deleted by e_i . While advancing/retreating we cannot look up a target record by its index. Instead, we maintain a second B-tree, mapping from each event's ID to the target record. The mapping stores a value depending on the type of the event:

- For delete events, we store the ID of the character deleted by the event.
- For insert events, we store a pointer to the leaf node in the first B-tree that contains the corresponding record.

When nodes in the first B-tree are split, we update the pointers in the second B-tree accordingly.

On every *apply*(e), after updating the sequence as above, we update this mapping. When we later call *retreat*(e) or *advance*(e), that event e must have already been applied, and hence $e.id$ must appear in this mapping. This map allows us to advance or retreat in logarithmic time.

3.5 Clearing the internal state

As described so far, the algorithm retains every insertion since document creation forever in its internal state, consuming a lot of memory, and requiring the entire event graph to be replayed in order to restore the internal state. We now introduce a further optimisation that allows Eg-walker to completely discard its internal state from time to time, and replay only a subset of the event graph.

We define a version $V \subseteq G$ to be a *critical version* in an event graph G iff it partitions the graph into two subsets of events $G_1 = \text{Events}(V)$ and $G_2 = G - G_1$ such that all events in G_1 happened before all events in G_2 :

$$\forall e_1 \in G_1 : \forall e_2 \in G_2 : e_1 \rightarrow e_2.$$

Equivalently, V is a critical version iff every event in the graph is either in V , or an ancestor of some event in V , or happened after *all* of the events in V :

$$\forall e_1 \in G : e_1 \in \text{Events}(V) \vee (\forall e_2 \in V : e_2 \rightarrow e_1).$$

A critical version might not remain critical forever; it is possible for a critical version to become non-critical because a concurrent event is added to the graph.

A key insight in the design of Eg-walker is that critical versions partition the event graph into sections that can be processed independently. Events that happened at or before a critical version do not affect how any event after the critical version is transformed. This observation enables two important optimisations:

- Any time the version of the event graph processed so far is critical, we can discard the internal state (including both B-trees and all s_p and s_e values), and replace it with a placeholder as explained in Section 3.6.
- If both an event's version and its parent version are critical versions, there is no need to traverse the B-trees and update the CRDT state, since we would immediately discard that state anyway. In this case, the transformed event is identical to the original event, so the event can simply be emitted as-is.

These optimisations make it very fast to process documents that are mostly edited sequentially (e.g., because the authors took turns and did not write concurrently, or because there is only a single author), since most of the event graph of such a document is a linear chain of critical versions.

The internal state can be discarded once replay is complete, although it is also possible to retain the internal state for transforming future events. If a replica receives events

that are concurrent with existing events in its graph, but the replica has already discarded its internal state resulting from those events, it needs to rebuild some of that state. It can do this by identifying the most recent critical version that happened before the new events, replaying the existing events that happened after that critical version, and finally applying the new events. Events from before that critical version are not replayed. Since most editing histories have critical versions from time to time, this means that usually only a small subset of the event graph is replayed. In the worst case, this algorithm replays the entire event graph.

3.6 Partial event graph replay

Assume that we want to add event e_{new} to the event graph G , that $V_{\text{curr}} = \text{Version}(G)$ is the current document version reflecting all events except e_{new} , and that $V_{\text{crit}} \neq V_{\text{curr}}$ is the latest critical version in $G \cup \{e_{\text{new}}\}$ that happened before both e_{new} and V_{curr} . Further assume that we have discarded the internal state, so the only information we have is the latest document state at V_{curr} and the event graph; in particular, without replaying the entire event graph we do not know the document state at V_{crit} .

Luckily, the exact internal state at V_{crit} is not needed. All we need is enough state to transform e_{new} and rebase it onto the document at V_{curr} . This internal state can be obtained by replaying the events since V_{crit} , that is, $G - \text{Events}(V_{\text{crit}})$, in topologically sorted order:

1. We initialise a new internal state corresponding to version V_{crit} . Since we do not know the the document state at this version, we start with a single placeholder record representing the unknown document content.
2. We update the internal state by replaying events from V_{crit} to V_{curr} , but we do not output transformed operations during this stage.
3. Finally, we apply the new event e_{new} and output the transformed operation. If we received a batch of new events, we apply them in topologically sorted order.

The placeholder record we start with in step 1 represents the range of indexes $[0, \infty]$ of the document state at V_{crit} (we do not know the length of the document at that version, but we can still have a placeholder for arbitrarily many indexes). Placeholders are counted as the number of characters they represent in the order statistic tree construction, and they have the same length in both the prepare and the effect versions. We then apply events as follows:

- Applying an insertion at index i creates a record with $s_p = s_e = \text{Ins}$ and the ID of the insertion event. We map the index to a record in the sequence using the prepare state as usual; if i falls within a placeholder for range $[j, k]$, we split it into a placeholder for $[j, i - 1]$, followed by the new record, followed by a placeholder for $[i, k]$. Placeholders for empty ranges are omitted.

- Applying a deletion at index i : if the deleted character was inserted prior to V_{crit} , the index must fall within a placeholder with some range $[j, k]$. We split it into a placeholder for $[j, i - 1]$, followed by a new record with $s_p = \text{Del}$ and $s_e = \text{Del}$, followed by a placeholder for $[i + 1, k]$. The new record has a placeholder ID that only needs to be unique within the local replica, and need not be consistent across replicas.
- Applying a deletion of a character inserted since V_{crit} updates the record created by the insertion.

Before applying an event we retreat and advance as usual. The algorithm never needs to retreat or advance an event that happened before V_{crit} , therefore every retreated or advanced event ID must exist in second B-tree.

If there are concurrent insertions at the same position, we invoke the CRDT algorithm to place them in a consistent order as discussed in Section 3.3. Since all concurrent events must be after V_{crit} , they are included in the replay. When we are seeking for the insertion position, we never need to seek past a placeholder, since the placeholder represents characters that were inserted before V_{crit} .

3.7 Algorithm complexity

Say we have two users who have been working offline, generating k and m events respectively. When they come online and merge their event graphs, the latest critical version is immediately prior to the branching point. If the branch of k events comes first in the topological sort, the replay algorithm first applies k events, then retreats k events, applies m events, and finally advances k events again. Asymptotically, $O(k + m)$ calls to apply/retreat/advance are required regardless of the order of traversal, although in practice the algorithm is faster if $k < m$ since we don't need to retreat/advance on the branch that is visited last.

Each apply/retreat/advance requires one or two traversals of first B-tree, and at most one traversal of the second B-tree. The upper bound on the number of entries in each tree (including placeholders) is $2(k + m) + 1$, since each event generates at most one new record and one placeholder split. Since the trees are balanced, the cost of each traversal is $O(\log(k + m))$. Overall, the cost of merging branches with k and m events is therefore $O((k + m) \log(k + m))$.

We can also give an upper bound on the complexity of replaying an arbitrary event graph with n events. Each event is applied exactly once, and before each event we retreat or advance each prior event at most once, at $O(\log n)$ cost. The worst-case complexity of the algorithm is therefore $O(n^2 \log n)$, but this case is unlikely to occur in practice.

3.8 Storing the event graph

To store the event graph compactly on disk, we developed a compression technique that takes advantage of how people typically write text documents: namely, they tend to insert

or delete consecutive sequences of characters, and less frequently hit backspace or move the cursor to a new location. Eg-walker’s event graph storage format is inspired by the Automerge CRDT library [27, 32], which in turn uses ideas from column-oriented databases [6, 55]. We also borrow some bit-packing tricks from the Yjs CRDT library [30].

We first topologically sort the events in the graph. Different replicas may sort the graph differently, but locally to one replica we can identify an event by its index in this sorted order. Then we store different properties of events in separate byte sequences called *columns*, which are then combined into one file with a simple header. Each column stores different fields of the event data. The columns are:

- *Event type, start position, and run length.* For example, “the first 23 events are insertions at consecutive indexes starting from index 0, the next 10 events are deletions at consecutive indexes starting from index 7,” and so on. We encode this using a variable-length binary encoding of integers, which represents small numbers in one byte, larger numbers in two bytes, etc.
- *Inserted content.* An insertion event contains exactly one character (a Unicode scalar value), and a deletion does not. We concatenate the UTF-8 encoding of the characters for insertion events in the same order as they appear in the first column, and LZ4-compress.
- *Parents.* By default we assume that every event has exactly one parent, namely its predecessor in the topological sort. Any events for which this is not true are listed explicitly, for example: “the first event has zero parents; the 153rd event has two parents, namely events 31 and 152;” and so on.
- *Event IDs.* Each event is uniquely identified by a pair of a replica ID and a per-replica sequence number. This column stores runs of event IDs, for example: “the first 1085 events are from replica A, starting with sequence number 0; the next 595 events are from replica B, starting with sequence number 0;” and so on.

Replicas can optionally also store a copy of the final document state reflecting all events. This allows documents to be loaded from disk without replaying the event graph.

We send the same data format over the network when replicating the entire event graph. When sending a subset of events over the network (e.g., a single event during real-time collaboration), references to parent events outside of that subset need to be encoded using event IDs of the form (*replicaID*, *seqNo*), but otherwise the encoding is similar.

4 Evaluation

We created a TypeScript implementation of Eg-walker optimised for simplicity and readability [24], and a production-ready Rust implementation optimised for performance [25]. The TypeScript version omits the run-length encoding of internal state, B-trees, and topological sorting heuristics.

To evaluate the correctness of Eg-walker we proved that the algorithm complies with Attiya et al.’s *strong list specification* [8] (see the extended version of this paper [26]). We also performed randomised property testing on the implementations, including checking that our implementations converge to the same result.

4.1 Editing traces

As there is no established benchmark for collaborative text editing, we collected a set of editing traces from real documents and made them freely available [23]. Statistics for these traces are given in Table 1. The traces represent the editing history of the following documents:

Sequential Traces: These traces have no concurrency.

Trace S1 is the LaTeX source of a journal paper [33, 34], S2 is an 8,800-word blog post [22], and S3 is the text of this paper that you are currently reading. S2 has one author; S1 and S3 have two authors who took turns.

Concurrent Traces: Trace C1 is two users collaboratively writing a reflection on TV series they have just watched. C2 is two users collaboratively reflecting on going to clown school together. We added 1 sec (C1) or 0.5 sec (C2) artificial latency between the users to increase the incidence of concurrent operations.

Asynchronous Traces: We reconstructed the editing trace of some files in Git repositories. The event graph mirrors the branching/merging of Git commits. Since Git does not record individual keystrokes, we generated the minimal edit operations necessary to perform each commit’s diff. Trace A1 is `src/node.cc` from the Git repository for Node.js [4], and A2 is `Makefile` from the Git repository for Git itself [3].

Even though the sequential traces do not exercise the merging algorithm, they are important to include in the benchmark. Anecdotal evidence suggests that the majority of documents in practice are sequentially edited – that is, they either have a single author, or multiple authors who take turns to write. The concurrent traces have many short-lived branches (see the *graph runs* column in Table 1). The asynchronous traces have a small number of long-running branches, which occur in the context of offline working or editors that support explicit branching and merging [40, 42].

We recorded the sequential and concurrent traces ourselves, collaborating with friends or colleagues, using an instrumented text editor that recorded keystroke-granularity editing events. All contributors to the traces have given their consent for their recorded keystroke data to be made publicly available and to be used for benchmarking purposes. The asynchronous traces are derived from public data on GitHub.

The recorded editing traces originally varied a great deal in length. To allow easier comparison of measurements between traces, we have normalised the length of the traces to contain approximately 500k inserted characters (except for

Table 1. The text editing traces used in our evaluation. *Repeats*: number of times the original trace was repeated to normalise its length relative to the other traces. *Events*: total number of editing events, in thousands, including repeats. Each inserted or deleted character counts as one event. *Average concurrency*: mean number of concurrent branches per event in the trace. *Graph runs*: number of sequential runs of events (linear event sequences without branching/merging). *Authors*: number of users who added at least one event. *Chars remaining*: percentage of inserted characters that remain in the document (i.e., are never deleted) after all events have been merged. *Final size*: resulting document size in kilobytes after all events have been merged.

Name	Type	Repeats	Events (k)	Avg Concurrency	Graph runs	Authors	Chars remaining (%)	Final size (kB)
S1	sequential	3	779	0.00	1	2	57.5	307.2
S2	sequential	3	1105	0.00	1	1	26.7	166.3
S3	sequential	1	2339	0.00	1	2	9.9	119.5
C1	concurrent	25	652	0.43	92101	2	90.1	521.5
C2	concurrent	25	608	0.44	133626	2	93.0	516.3
A1	asynchronous	1	947	0.10	101	194	7.8	37.2
A2	asynchronous	2	698	6.11	2430	299	49.6	222.0

S3, which is approximately twice this size). We did this by repeating the original S1 and S2 traces 3 times, the original C1 and C2 traces 25 times, and the original A2 trace twice. The statistics given in Table 1 are after repetition.

4.2 Experimental approach

To evaluate the performance of Eg-walker, we compare our Rust implementation with two popular CRDT libraries: Automerge v0.5.9 [1] (Rust) and Yjs v13.6.10 [30] (JavaScript).¹ We only test their collaborative text datatypes, and not the other features they support. However, the performance of these libraries varies widely. In an effort to distinguish between implementation differences and algorithmic differences, we have also implemented our own performance-optimised reference CRDT library. This library shares most of its code with our Rust Eg-walker implementation, enabling a more like-to-like comparison between the traditional CRDT approach and Eg-walker. Our reference CRDT outperforms both Yjs and Automerge.

We have also implemented a simple OT library using the TTF algorithm [46]. We do not use the server-based Jupiter algorithm [44] or the popular OT library ShareDB [21] because they do not support the branching and merging patterns that occur in our asynchronous traces.

We compare these implementations along 3 dimensions:²

Speed: The CPU time to load a document into memory, and to merge a set of updates from a remote replica.

Memory usage: The RAM used while a document is loaded and while merging remote updates.

Storage size: The number of bytes needed to persistently store a document or send it over the network.

By design, our experiments do not run over a network, but focus on single-node CPU and memory use (along with storage size, which is also a measure of network bandwidth used). We made this choice because memory use and loading time are the biggest challenges with CRDTs, and CPU time on long-running branches is the biggest challenge with OT. We delegate replication to the reliable broadcast protocol, which is beyond the scope of this paper. Moreover, the results in a distributed setup would depend highly on the online/offline pattern we assume; running on a single node allows us to more directly compare the algorithms.

4.3 Time taken to load and merge changes

The slowest operations in many collaborative editors are:

- merging a large set of edits from a remote replica into the local state (e.g. reconnecting after working offline);
- loading a document from disk into memory so that it can be displayed and edited.

To simulate a worst-case merge, we start with an empty document and then merge an entire editing trace into it. In the case of Eg-walker this means replaying the full trace. Figure 8 shows the merge time for each implementation. Such a large merge occurs in practice when a node has been offline for a long time and needs to catch up on most of the editing history, or when replaying a subset of the editing history in order to reconstruct a historical version of the document (for history visualisation). Moreover, in the CRDT implementations we tested, loading a document from disk takes the same CPU time as merging all of the events. Smaller merges, which occur during online collaboration, are faster.

¹We also tested Yrs [58], the Rust rewrite of Yjs by the original authors. At the time of our experiments it performed worse than Yjs, so we omitted it from our results.

²Experimental setup: We ran the benchmarks on a Ryzen 7950x CPU running Linux 6.5.0-28 and 64GB of RAM. We compiled Rust code with rustc v1.78.0 in release mode with “-C target-cpu=native”. Rust code was pinned to a single CPU core to reduce variance across runs. For JavaScript (Yjs) we used Node.js v22.2.0. All reported time measurements are the mean of at least 100 test iterations (except for the case where OT takes an hour to merge trace A2, which we ran 10 times). The standard deviation for all benchmark results was less than 1.2% of the mean, except for the Yjs measurements, which had a stddev of less than 6%. Error bars on our graphs are too small to be visible.

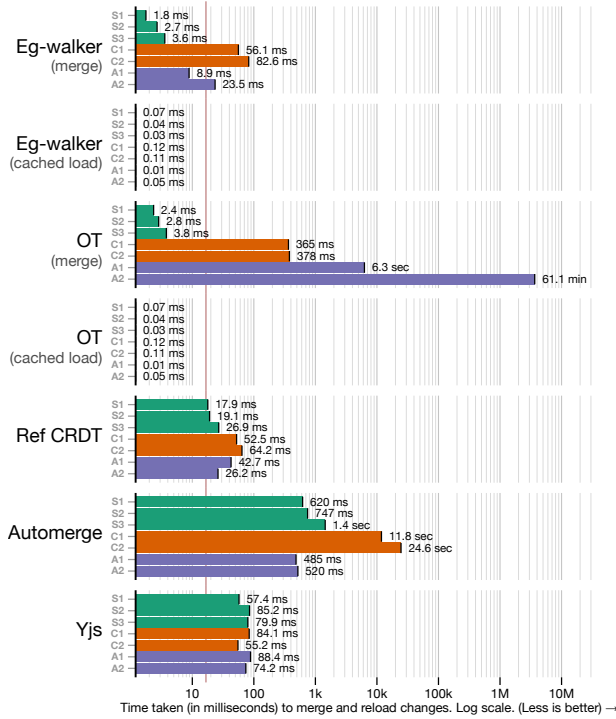


Figure 8. The CPU time taken by each algorithm to merge all events in each trace (as received from a remote replica), or to reload the resulting document from disk. The CRDT implementations (Ref CRDT, Automerger and Yjs) take the same amount of time to merge changes as they do to subsequently load the document. The red line at 16 ms indicates the time budget available to an application that wants to show the results of an operation by the next frame, assuming a display with a 60 Hz refresh rate.

After merging the entire trace, we save the resulting local replica state to disk and measure the CPU time to load it back into memory. Since loading and merging take the same time in the CRDTs we tested, we do not show their loading times separately in Figure 8. In these algorithms, the CRDT metadata needs to be in memory for the user to be able to edit the document, or to apply any updates received from other replicas (even when there is no concurrency). In contrast, OT and Eg-walker can load documents orders of magnitude faster than CRDTs by caching the final document state on disk, and loading just this data (essentially a plain text file). Eg-walker and OT only need to load the event graph when merging concurrent changes or to reconstruct old document versions. Document edits by the local user or applying non-concurrent remote events do not need the event graph.

We can see in Figure 8 that Eg-walker and OT are very fast to merge the sequential traces (S1, S2, S3), since they simply apply the operations with no transformation. However,

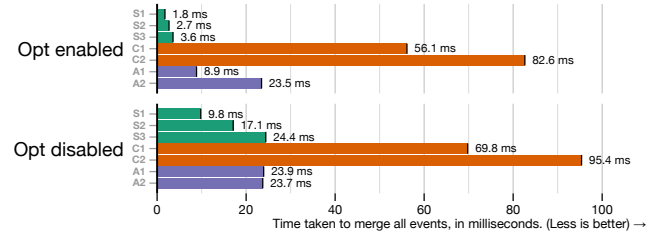


Figure 9. Time taken for Eg-walker to merge all events in a trace, with and without the optimisations from Section 3.5.

OT performance degrades dramatically on the asynchronous traces (6 seconds for A1, and 1 hour for A2) due to the quadratic complexity of the algorithm, whereas Eg-walker remains fast (160,000× faster in the case of A2).

On the concurrent traces (C1, C2) and asynchronous trace A2, the merge time of Eg-walker is similar to that of our reference CRDT, since they perform similar work. Both are significantly faster than the state-of-the-art Yjs and Automerger CRDT libraries; this is due to implementation differences and not fundamental algorithmic reasons.

On the sequential traces Eg-walker outperforms our reference CRDT by a factor of 7–10×, and on trace A1 (which contains large sequential sections) Eg-walker is 5× faster. Comparing to Yjs or Automerger, this speedup is greater still. This is due to Eg-walker’s ability to clear its internal state and skip all of the internal state manipulation on critical versions (Section 3.5). To quantify this effect, Figure 9 compares the time taken to replay all our traces with this optimisation enabled and disabled. We see that the optimisation is effective for S1, S2, S3, and A1, whereas for C1, C2, and A2 it makes little difference (A2 contains no critical versions).

When merging an event graph with very high concurrency (like A2), the performance of Eg-walker is highly dependent on the order in which events are traversed. A poorly chosen traversal order can make this trace as much as 8× slower to merge. Our topological sort algorithm (Section 3.2) tries to avoid such pathological cases.

4.4 RAM usage

Figure 10 shows the memory footprint (retained heap size) of each algorithm. For Eg-walker and OT it shows both peak usage (while replaying the entire editing trace) and “steady state” memory usage (after temporary data and Eg-walker’s internal state are discarded and the event graph is written out to disk). For the CRDTs the figure shows steady state memory usage; peak usage is up to 25% higher.

Eg-walker’s peak memory use is similar to our reference CRDT’s steady state: slightly lower on the sequential traces, and approximately double for the concurrent traces. However, the steady-state memory use of Eg-walker is 1–2 orders

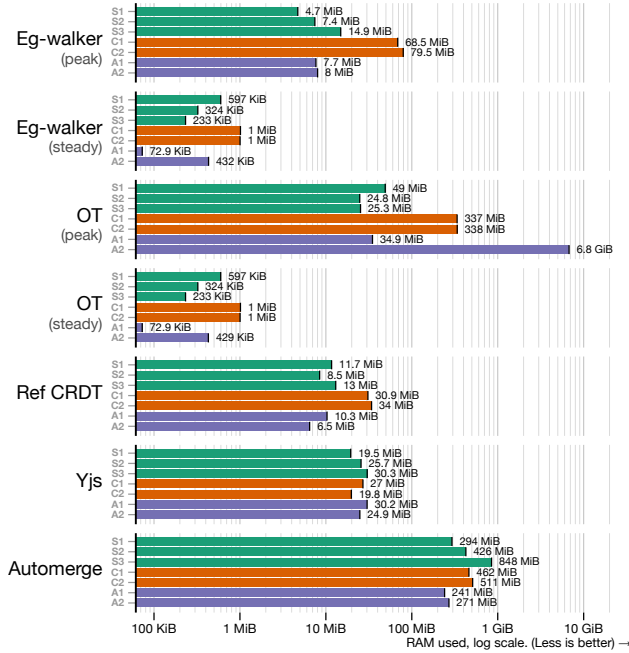


Figure 10. RAM used while merging an editing trace received from another replica. Eg-walker and OT only retain the current document text in the steady state, but need additional RAM at peak while merging concurrent changes.

of magnitude lower than the best CRDT. This is a significant result, since the steady state is what matters during normal operation while a document is being edited. Memory usage reaches this peak only when replaying the entire trace; it is lower when merging a small branch. Yjs has up to a 3× greater memory use than our reference CRDT, and Automerge an order of magnitude greater.

OT has the same memory use as Eg-walker in the steady state, but significantly higher peak memory use on the C1, C2, and A2 traces (6.8 GiB for A2). The reason is that our OT implementation memoizes intermediate transformed operations to improve performance. This memory use could be reduced at the cost of increased merge times. The computer we used for benchmarking had enough RAM to prevent swapping in all cases.

4.5 Storage size

Our binary encoding of event graphs (Section 3.8) results in smaller files than the equivalent internal CRDT state persisted by Automerge, and in many cases, Yjs. To ensure a like-for-like comparison we have disabled Eg-walker’s built-in LZ4 and Automerge’s built-in gzip compression. Enabling this compression further reduces the file sizes.

Automerge stores the full editing history of a document, and Figure 11 shows the resulting file sizes relative to the raw concatenated text content of all insertions, with and

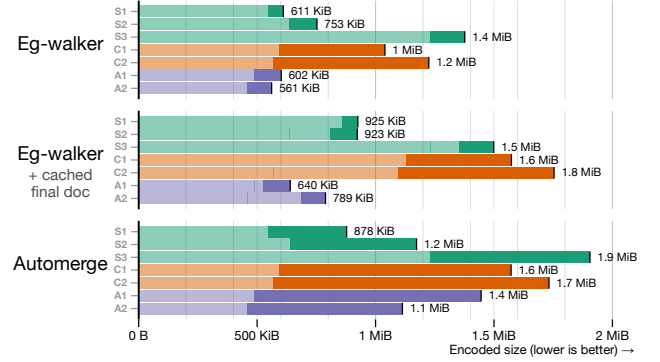


Figure 11. File size storing edit traces using Eg-walker’s event graph encoding (with and without final document caching) compared to Automerge. The lightly shaded region in each bar shows the concatenated length of all stored text. This acts as lower bound on the file size.

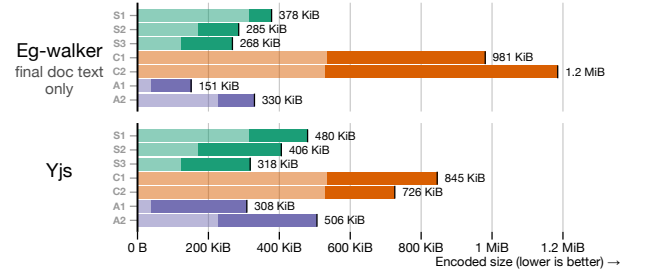


Figure 12. File size storing edit traces in which deleted text content has been omitted, as is the case with Yjs. The lightly shaded region in each bar is the size of the final document, which is a lower bound on the file size.

without a cached copy of the final document state (to enable fast loads).

In contrast, Yjs only stores the resulting document text, and any data needed to merge changes. Yjs does not store deleted characters or the happened-before relationship between events. Figure 12 compares Yjs to the equivalent event graph encoding in which we only store the final document text and operation metadata. Our encoding is smaller than Yjs on the sequential and async traces, but larger for the concurrent traces, where the edges in the event graph take more space. The overhead of storing the event graph is between 20% and 3× the final plain text file size.

5 Related Work

Eg-walker is an example of a *pure operation-based CRDT* [9], which is a family of algorithms that capture a DAG (or partially ordered log) of operations in the form they were generated, and define the current state as a query over that log. However, existing publications on pure operation-based

CRDTs [7, 10] present only datatypes such as maps, sets, and registers; Eg-walker adds a list/text datatype to this family.

MRDTs [54] are similarly based on a DAG, and use a three-way merge function to combine two branches since their lowest common ancestor; if the LCA is not unique, a recursive merge is used. MRDTs for various datatypes have been defined, but so far none offers text with arbitrary insertion and deletion.

Toomim’s *time machines* approach [59] shares a conceptual foundation with Eg-walker: both are based on traversing an event graph, with operations being transformed from their original form into a form that can be applied in topologically sorted order. Toomim also points out that CRDTs can implement this transformation. Eg-walker is a concrete, optimised implementation of the time machine approach; novel contributions of Eg-walker include updating the prepare version by retreating and advancing, as well as the details of internal state clearing and partial event graph replay.

Eg-walker is also an *operational transformation* (OT) algorithm [18]. OT has a long lineage of research going back to the 1990s [44, 51, 56]. To our knowledge, all existing OT algorithms consist of a set of *transformation functions* that transform one operation with regard to one other operation, and a *control algorithm* that traverses an editing history and invokes the necessary transformations. A problem with this architecture is that when two replicas have diverged and each performed n operations, merging their states unavoidably has a cost of at least $O(n^2)$; in some OT algorithms the cost is cubic or even worse [39, 52, 57]. Eg-walker departs from the transformation function/control algorithm architecture and instead performs transformations using an internal CRDT state, which reduces the merging cost to $O(n \log n)$ in most cases; the upper bound of $O(n^2 \log n)$ is unlikely to occur in practical editing histories.

Other collaborative text editing algorithms [49, 52, 60, 61] belong to the family of *conflict-free replicated data types* (CRDTs) [53]. To our knowledge, all existing CRDTs for text work by assigning each character a unique ID, and translating index-based insertions and deletions into ID-based ones. These unique IDs need to be held in memory when a document is being edited, persisted for the lifetime of the document, and sent to all replicas. In contrast, Eg-walker uses unique IDs only transiently during replay but does not persist or replicate them, and it can free its internal state whenever a critical version is reached. Eg-walker needs to store the event graph as long as concurrent operations may arrive, but this takes less space than CRDT state, and it only needs to be in-memory while merging concurrent operations. Most of the time the event graph can remain on disk.

Gu et al.’s mark & retrace method [28] builds a CRDT-like structure containing the entire editing history, not only the parts being merged. Differential synchronization [20] relies on heuristics such as similarity-matching of text to perform merges, which is not guaranteed to converge.

Version control systems such as Git [14], Pijul [43], and Darcs [2] also track the editing history of text files. However, they do not support real-time collaboration, and they are line-based (good for code), whereas Eg-walker is character-based (which is better for prose). Git uses a three-way merge, which is not reliable on files containing substantial repeated text [31]. Merges in Darcs have worst-case exponential complexity [37], and Pijul merges using a CRDT that assigns a unique ID to every line [5].

6 Conclusion

Eg-walker is a new approach to collaborative text editing that has characteristics of both CRDTs and OT. It is orders of magnitude faster than existing algorithms in the best cases, and competitive with the fastest existing implementations in the worst cases. Compared to existing CRDTs, it uses orders of magnitude less memory in the steady state, files are vastly faster to load for editing, and in documents with largely sequential editing edits from other users are merged much faster. Compared to OT, large merges (e.g., when two users each did a significant amount of work while offline) are much faster, and Eg-walker supports arbitrary branching/merging patterns (e.g., in peer-to-peer collaboration).

Since Eg-walker stores a fine-grained editing history of a document, it allows applications to show that history to the user, and to restore arbitrary past versions of a document by replaying subsets of the graph. The underlying event graph is not specific to the Eg-walker algorithm, so we expect that the same data format will be able to support future collaborative editing algorithms as well. The core idea of Eg-walker is not specific to plain text; we believe it can be extended to other file types such as rich text, graphics, or spreadsheets.

Until now, many applications have been implemented using centralised server-based OT to avoid the overheads of CRDTs. Eg-walker is the first CRDT to match OT’s memory use and performance on sequential editing histories (which are common in practice), while avoiding the quadratic merge complexity that makes OT impractical for long-running branches. By requiring no server, Eg-walker makes it possible for decentralised, local-first software [36] to become competitive with traditional cloud software.

Acknowledgments

This work was made possible by generous support from Michael Toomim, the Braid community and the Invisible College. None of this would have happened without their help. Thank you for the endless conversations we have shared about collaborative editing. Martin Kleppmann gratefully acknowledges his crowdfunding supporters including Minter and SoftwareMill. Thank you to Matthew Weidner, Joe Hellerstein, and our shepherd Diogo Behrens for feedback that helped improve this paper.

References

- [1] [n. d.]. *Automerger CRDT*. <https://automerger.org/>
- [2] [n. d.]. *Darcs*. <https://darcs.net/>
- [3] [n. d.]. *Makefile for Git*. <https://github.com/git/git/blob/master/Makefile>
- [4] [n. d.]. *Node.js source code: src/node.cc*. <https://github.com/nodejs/node/blob/main/src/node.cc>
- [5] [n. d.]. *The Pijul manual: Theory*. <https://pijul.org/manual/theory.html> Archived at <https://perma.cc/NAU4-SMYZ>.
- [6] Daniel J Abadi, Peter Boncz, Stavros Harizopoulos, Stratos Idreos, and Samuel Madden. 2013. The Design and Implementation of Modern Column-Oriented Database Systems. *Foundations and Trends in Databases* 5, 3 (2013), 197–280. <https://doi.org/10.1561/19000000024>
- [7] Paulo Sérgio Almeida. 2023. Approaches to Conflict-free Replicated Data Types. (Oct. 2023). arXiv:2310.18220 <https://arxiv.org/abs/2310.18220>
- [8] Hagit Attiya, Sebastian Burckhardt, Alexey Gotsman, Adam Morrison, Hongseok Yang, and Marek Zawirski. 2016. Specification and Complexity of Collaborative Text Editing. In *ACM Symposium on Principles of Distributed Computing (PODC 2016)*. 259–268. <https://doi.org/10.1145/2933057.2933090>
- [9] Carlos Baquero, Paulo Sergio Almeida, and Ali Shoker. 2017. Pure Operation-Based Replicated Data Types. (2017). <https://arxiv.org/abs/1710.04469>
- [10] Jim Bauwens and Elisa Gonzalez Boix. 2023. Nested Pure Operation-Based CRDTs. In *37th European Conference on Object-Oriented Programming (ECOOP 2023)*. Schloss Dagstuhl, 2:1–2:26. <https://doi.org/10.4230/LIPIcs.ECOOP.2023.2>
- [11] Kenneth Birman, André Schiper, and Pat Stephenson. 1991. Lightweight causal and atomic group multicast. *ACM Transactions on Computer Systems* 9, 3 (Aug. 1991), 272–314. <https://doi.org/10.1145/128738.128742>
- [12] Hans-J. Boehm, Russ Atkinson, and Michael Plass. 1995. Ropes: An alternative to strings. *Software Prac. Experience* 25, 12 (1995), 1315–1330. <https://doi.org/10.1002/spe.4380251203>
- [13] Christian Cachin, Rachid Guerraoui, and Luís Rodrigues. 2011. *Introduction to Reliable and Secure Distributed Programming* (second ed.). Springer.
- [14] James Coglan. 2019. *Building Git*. <https://shop.jcoglan.com/building-git/>
- [15] Paul Coldren. 2024. *Engineering for Slow Internet: How to minimize user frustration in Antarctica*. <https://brr.fyi/posts/engineering-for-slow-internet> Archived at <https://perma.cc/7Q9B-TUJV>.
- [16] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. 2009. *Introduction to Algorithms* (third ed.). MIT Press.
- [17] John Day-Richter. 2010. *What's different about the new Google Docs: Making collaboration fast*. <https://drive.googleblog.com/2010/09/whats-different-about-new-google-docs.html> Archived at <https://perma.cc/5FVM-542B>.
- [18] C A Ellis and S J Gibbs. 1989. Concurrency control in groupware systems. In *ACM International Conference on Management of Data (SIGMOD 1989)*. 399–407. <https://doi.org/10.1145/67544.66963>
- [19] Aaron Fabbri. 2024. *How Ditto Empowers the U.S. Navy's Unmanned Vehicle Operations, Keeping America Safe*. <https://ditto.live/blog/how-ditto-empowers-the-u-s-navy-s-unmanned-vehicle-operations-keeping-america-safe> Archived at <https://perma.cc/UAM9-JX5C>.
- [20] Neil Fraser. 2009. Differential synchronization. In *9th ACM Symposium on Document Engineering (DocEng 2009)*. ACM, 13–20. <https://doi.org/10.1145/1600193.1600198>
- [21] Joseph Gentle. 2021. *ShareDB*. <https://github.com/share/sharedb>
- [22] Joseph Gentle. 2021. *5000x faster CRDTs: An Adventure in Optimization*. <https://josephg.com/blog/crdts-go-brrr/>
- [23] Joseph Gentle. 2023. *Editing Traces (github repository)*. <https://github.com/josephg/editing-traces>
- [24] Joseph Gentle. 2023. *Reference Eg-walker implementation in Typescript*. <https://github.com/josephg/eg-walker-reference>
- [25] Joseph Gentle. 2024. *Diamond Types: A fully featured realtime editing library*. <https://github.com/josephg/diamond-types>
- [26] Joseph Gentle and Martin Kleppmann. 2024. Collaborative Text Editing with Eg-walker: Better, Faster, Smaller. Extended version. (Sept. 2024). arXiv:2409.14252 <https://arxiv.org/abs/2409.14252>
- [27] Alex Good and Andrew Jeffery. [n. d.]. *Automerger Binary Document Format*. <https://automerger.org/automerger-binary-format-spec/> Archived at <https://perma.cc/XV25-RA4U>.
- [28] Ning Gu, Jiangming Yang, and Qiwei Zhang. 2005. Consistency maintenance based on the mark & retrace technique in groupware systems. In *ACM International Conference on Supporting Group Work (GROUP 2005)*. ACM, 264–273. <https://doi.org/10.1145/1099203.1099250>
- [29] Joseph M Hellerstein. 2010. The Declarative Imperative: Experiences and Conjectures in Distributed Logic. *ACM SIGMOD Record* 39, 1 (Sept. 2010), 5–19. <https://doi.org/10.1145/1860702.1860704>
- [30] Kevin Jahns. [n. d.]. *Yjs Shared Editing*. <https://yjs.dev/>
- [31] Sanjeev Khanna, Keshav Kunal, and Benjamin C Pierce. 2007. A Formal Investigation of Diff3. In *27th International Conference on Foundations of Software Technology and Theoretical Computer Science (FSTTCS 2007)*. Springer, 485–496. https://doi.org/10.1007/978-3-540-77050-3_40
- [32] Martin Kleppmann. 2019. *Experiment: columnar data encoding for Automerger*. <https://github.com/automerger/automerger-perf/blob/master/columnar/README.md> Archived at <https://perma.cc/57KC-PP4Y>.
- [33] Martin Kleppmann. 2020. *Benchmarking resources for Automerger*. <https://github.com/automerger/automerger-perf>
- [34] Martin Kleppmann and Alastair R Beresford. 2017. A Conflict-Free Replicated JSON Datatype. *IEEE Transactions on Parallel and Distributed Systems* 28, 10 (April 2017), 2733–2746. <https://doi.org/10.1109/TPDS.2017.2697382> arXiv:1608.03960
- [35] Martin Kleppmann, Annette Bieniusa, and Marc Shapiro. [n. d.]. *CRDT Papers*. <https://crdt.tech/papers.html>
- [36] Martin Kleppmann, Adam Wiggins, Peter van Hardenberg, and Mark McGranaghan. 2019. Local-First Software: You own your data, in spite of the cloud. In *ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward! 2019)*. ACM, 154–178. <https://doi.org/10.1145/3359591.3359737>
- [37] Eric Kow. [n. d.]. *Understanding Darcs*. https://en.wikibooks.org/wiki/Understanding_Darcs/Print_Version Archived at <https://perma.cc/3VZF-8J65>.
- [38] Leslie Lamport. 1978. Time, clocks, and the ordering of events in a distributed system. *Commun. ACM* 21, 7 (1978), 558–565. <https://doi.org/10.1145/359545.359563>
- [39] Du Li and Rui Li. 2006. A performance study of group editing algorithms. In *12th International Conference on Parallel and Distributed Systems (ICPADS 2006)*. <https://doi.org/10.1109/icpads.2006.18>
- [40] Geoffrey Litt, Paul Sonnentag, Max Schöning, Adam Wiggins, Peter van Hardenberg, and Orion Henry. 2024. *Patchwork lab notebook: Version control for everything*. Technical Report. Ink & Switch. <https://www.inkandswitch.com/patchwork/notebook/> Archived at <https://perma.cc/VJM7-YPJB>.
- [41] Peng Lyu. 2018. *Text Buffer Reimplementation*. <https://code.visualstudio.com/blogs/2018/03/23/text-buffer-reimplementation> Archived at <https://perma.cc/V695-K7EL>.
- [42] Karissa Rae McKelvey, Scott Jenson, Eileen Wagner, Blaine Cook, and Martin Kleppmann. 2023. *Upwelling: Combining real-time collaboration with version control for writers*. Technical Report. Ink & Switch. <https://www.inkandswitch.com/upwelling/> Archived at <https://perma.cc/7TT8-X8S9>.
- [43] Pierre-Étienne Meunier and Florent Becker. [n. d.]. *Pijul*. <https://pijul.org/>

- [44] David A Nichols, Pavel Curtis, Michael Dixon, and John Lamping. 1995. High-latency, low-bandwidth windowing in the Jupiter collaboration system. In *8th Annual ACM Symposium on User Interface and Software Technology (UIST 1995)*. 111–120. <https://doi.org/10.1145/215585.215706>
- [45] Petru Nicolaescu, Kevin Jahns, Michael Derntl, and Ralf Klamma. 2016. Near Real-Time Peer-to-Peer Shared Editing on Extensible Data Types. In *19th International Conference on Supporting Group Work (GROUP 2016)*. ACM, 39–49. <https://doi.org/10.1145/2957276.2957310>
- [46] Gérald Oster, Pascal Molli, Pascal Urso, and Abdessamad Imine. 2006. Tombstone Transformation Functions for Ensuring Consistency in Collaborative Editing Systems. In *9th IEEE International Conference on Collaborative Computing (CollaborateCom 2006)*. <https://doi.org/10.1109/colcom.2006.361867>
- [47] Gérald Oster, Pascal Urso, Pascal Molli, and Abdessamad Imine. 2006. Data consistency for P2P collaborative editing. In *ACM Conference on Computer Supported Cooperative Work (CSCW 2006)*. 259–268. <https://doi.org/10.1145/1180875.1180916>
- [48] Overleaf. [n. d.]. *Can multiple authors edit the same file at the same time?* https://www.overleaf.com/learn/how-to/Can_multiple_authors_edit_the_same_file_at_the_same_time%3F Archived at <https://perma.cc/P8TH-YRMX>.
- [49] Nuno Preguiça, Joan Manuel Marques, Marc Shapiro, and Mihai Letia. 2009. A Commutative Replicated Data Type for Cooperative Editing. In *29th IEEE International Conference on Distributed Computing Systems (ICDCS 2009)*. 395–403. <https://doi.org/10.1109/icdcs.2009.20>
- [50] Ryan Ratner. 2024. *ANA Elevates Onboard Passenger Experience with Ditto*. <https://ditto.live/blog/ana-elevates-onboard-passenger-experience-with-ditto> Archived at <https://perma.cc/M6NG-AWLM>.
- [51] Matthias Ressel, Doris Nitsche-Ruhland, and Rul Gunzenhäuser. 1996. An integrating, transformation-oriented approach to concurrency control and undo in group editors. In *ACM Conference on Computer Supported Cooperative Work (CSCW 1996)*. 288–297. <https://doi.org/10.1145/240080.240305>
- [52] Hyun-Gul Roh, Myeongjae Jeon, Jin-Soo Kim, and Joonwon Lee. 2011. Replicated Abstract Data Types: Building Blocks for Collaborative Applications. *J. Parallel and Distrib. Comput.* 71, 3 (March 2011), 354–368. <https://doi.org/10.1016/j.jpdc.2010.12.006>
- [53] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. 2011. Conflict-free Replicated Data Types. In *13th International Conference on Stabilization, Safety, and Security of Distributed Systems (SSS 2011)*. 386–400. https://doi.org/10.1007/978-3-642-24550-3_29
- [54] Vimala Soundarapandian, Adharsh Kamath, Kartik Nagar, and KC Sivaramakrishnan. 2022. Certified mergeable replicated data types. In *43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI 2022)*. ACM, 332–347. <https://doi.org/10.1145/3519939.3523735>
- [55] Michael Stonebraker, Daniel J Abadi, Adam Batkin, Xuedong Chen, Mitch Cherniack, Miguel Ferreira, Edmond Lau, Amerson Lin, Samuel Madden, Elizabeth O’Neil, Patrick O’Neil, Alexander Rasin, Nga Tran, and Stanley Zdonik. 2005. C-Store: A Column-oriented DBMS. In *31st International Conference on Very Large Data Bases (VLDB 2005)*. 553–564.
- [56] Chengzheng Sun, Xiaohua Jia, Yanchun Zhang, Yun Yang, and David Chen. 1998. Achieving convergence, causality preservation, and intention preservation in real-time cooperative editing systems. *ACM Transactions on Computer-Human Interaction* 5, 1 (March 1998), 63–108. <https://doi.org/10.1145/274444.274447>
- [57] David Sun, Chengzheng Sun, Agustina Ng, and Weiwei Cai. 2020. Real Differences between OT and CRDT in Correctness and Complexity for Consistency Maintenance in Co-Editors. *Proceedings of the ACM on Human-Computer Interaction* 4, CSCW1, Article 21 (May 2020), 30 pages. <https://doi.org/10.1145/3392825>
- [58] Bartosz Sypytkowski, Kevin Jahns, and John Waidhofer. [n. d.]. *Y CRDT: Rust port of Yjs*. <https://github.com/y-crdt/y-crdt>
- [59] Michael Toomim. 2024. CRDT and OT generalize as Time Machines. (2024). <https://braid.org/time-machines> Archived at <https://perma.cc/VND4-ACEK>.
- [60] Matthew Weidner and Martin Kleppmann. 2023. The Art of the Fugue: Minimizing Interleaving in Collaborative Text Editing. (2023). <https://arxiv.org/abs/2305.00583>
- [61] Stéphane Weiss, Pascal Urso, and Pascal Molli. 2010. Logoot-Undo: Distributed Collaborative Editing System on P2P Networks. *IEEE Transactions on Parallel and Distributed Systems* 21, 8 (2010), 1162–1174. <https://doi.org/10.1109/tpds.2009.173>

A Artifact Appendix

A.1 Abstract

Our artifact is an implementation of the Eg-walker algorithm, together with the datasets and benchmarking tools required to run the experiments described in the paper. Specifically, it contains the following items:

- Editing traces of text documents that we use for benchmarking (see Table 1), in our own binary format and as JSON
- Source code for the following tools:
 - Our optimised Eg-walker implementation, written in Rust
 - Our reference CRDT implementation
 - Our reference OT implementation
 - Tools to convert our editing traces to JSON, and to the Yjs and Automerge file formats
 - Benchmarking tools to reproduce all experiments
- The tools to generate the figures in this paper

A.2 Description & Requirements

A.2.1 How to access. All code and data of the artifact is publicly available in the following GitHub repository:

<https://github.com/josephg/egwalker-paper>

A snapshot of this repository is archived at:

<https://zenodo.org/records/13823409>

doi:10.5281/zenodo.13823409

The README file in this repository contains detailed instructions to configure & run our code locally.

A.2.2 Hardware dependencies. To run our experiments, you need the following:

- A computer running Linux. Our software should work on a number of other systems including Windows and MacOS, but we have not tested on other platforms.
- At least 8GB of RAM, but 16GB is recommended. This is mainly required for the OT/A2 benchmark, which at peak uses approximately 7GB of RAM.
- Plenty of disk space. The Rust compiler produces a lot of temporary files – 44GB on our system.

A.2.3 Software dependencies. You need a recent compiler and runtime for the Rust programming language. We have tested with Rust 1.78; newer versions should also work.

You also need NodeJS installed in order to run the Yjs benchmark and to generate the charts showing our results. We used version 21.

Other software dependencies are managed through Cargo and npm respectively, and they are automatically installed as part of the build process.

A.2.4 Benchmarks. The datasets used by our benchmarks are included in the `datasets/` folder of our artifact repository, and the code to run the benchmarks is in the `tools/`

folder. Each of these folders contains a `README.md` file that documents its contents in more detail.

A.3 Set-up

Install Rust (the easiest way is via `rustup`) and NodeJS.

A.4 Evaluation workflow

Please see the `README.md` file in the artifact repository for a detailed description of the process.

A.4.1 Major Claims.

- C1.** Eg-walker is competitive with existing state-of-the-art collaborative editing systems in terms of file size, memory usage, and CPU time taken to merge changes. That is, on all our benchmarks, Eg-walker has at most $\approx 2\times$ the cost of the other systems we test, and in some cases dramatically lower cost. Experiments E1, E2, E3 and Figures 8–12 support this claim.
- C2.** On editing traces with long-running branches, the merge performance of Eg-walker is several orders of magnitude faster than OT, and slightly faster than the best CRDT implementations. This claim is supported by Experiment E3 and the data in Figure 8.
- C3.** On sequential editing traces, Eg-walker is as fast as OT, and about an order of magnitude faster than our reference CRDT (which in turn is the fastest among the CRDTs we tested). This is supported by Experiment E3 and the data in Figure 8. This is in large part due to the optimisations in Section 3.5, as shown in Figure 9.
- C4.** The time to load a document from disk (to view and edit) with Eg-walker is the same as with OT, which is several orders of magnitude faster than all CRDTs we tested. Experiment E3 and Figure 8 also show this.
- C5.** The steady-state memory consumption of Eg-walker is at least an order of magnitude lower than that of CRDTs, and the same as OT. The peak memory consumption of Eg-walker during merging is similar to the steady state of CRDTs, and lower than OT. This claim is supported by Experiment E2 and the data in Figure 10.

A.4.2 Experiment E1: File sizes (4 compute-hours).

Converts the raw editing traces in the `datasets/raw` directory into the Yjs, Automerge, and Eg-walker file formats (`datasets/*.{yjs,am,dt}`). Some of the traces are repeated several times so that they have a similar size (see Section 4.1). The sizes of the resulting files are reported in Figure 11 and Figure 12. The files also form an input to the subsequent experiments. The output files are included with the artifact so that the later experiments can be run without having to run this one.

To run this experiment:

```
rm datasets/*
./step1-prepare.sh
```

The resulting files should be byte-for-byte identical to those in the artifact. This experiment is slow because we have not made much effort to optimise it.

To generate the figures:

```
node collect.js
cd svg-plot
npm i # only needed once to install dependencies
node render.js
```

which writes Figure 11 to `diagrams/filesize_full.svg` and Figure 12 to `diagrams/filesize_smol.svg`.

A.4.3 Experiment E2: Memory use (1 compute-hour).

This experiment measures memory use – both the peak memory use while replaying each editing trace, and the “steady state” memory use once the replay is complete (keeping in memory the structures that are needed to display and edit a document, but freeing the structures that are needed only for replay and merging concurrent edits). We test with the following algorithms:

- Diamond-types (DT): our optimised Eg-walker implementation
- DT-CRDT: our reference CRDT implementation
- Automerge [1]
- Yjs [30]
- Yrs, the port of Yjs to Rust by the original authors [58]
- OT: our reference OT implementation

To run this experiment:

```
./step2a-memusage.sh
```

Then use `collect.js` and `render.js` like in E1 to generate Figure 10 (`diagrams/memusage.svg`). The data appears in `results/*_memusage.json`.

Note: Our OT implementation takes 1 hour to replay the A2 editing trace.

A.4.4 Experiment E3: Merge time (12 compute-hours).

This experiment measures the CPU time taken to merge the entire editing trace into an empty document, as if it had been received from a remote peer. The benchmarks cover the same algorithms as Experiment E2. To run this experiment:

```
./step2b-benchmarks.sh
```

Then use `collect.js` and `render.js` like in E1 to generate the charts for Figure 8 (`diagrams/timings.svg`) and Figure 9 (`diagrams/ff.svg`). The script writes summary data to `results/timings.json`.

A.5 Notes on Reusability

While our artifact contains a snapshot of our Eg-walker implementation at the time of this paper was written, the ongoing development of the implementation is part of the Diamond Types project in the following repository:

<https://github.com/josephg/diamond-types>

Diamond Types is freely available under the ISC license.

As part of this work, we have also created several editing traces of real text documents, as described in Section 4.1. The intention is that these traces can be used in future research for benchmarking collaborative text editing systems. The `datasets/` directory of the artifact contains those traces in JSON format. Additional traces that we may collect in the future will be added to the following repository:

<https://github.com/josephg/editing-traces>

That repository also contains documentation of the JSON-based file format that we use to encode the editing traces.

If you want to benchmark a CRDT using these editing traces, you need to convert them to your CRDT’s local format. We do this by simulating (in memory) a set of collaborating peers. The peers fork and merge their changes. The `tools/crdt-converter` directory of our artifact contains code to perform this process using Automerge and Yjs (Yrs). We believe this algorithm could be adapted to support most existing CRDT formats and systems.

A.6 General Notes

The performance of collaborative editing systems varies by orders of magnitude depending on how the implementation has been optimised. For example, early CRDTs were widely thought to be impractical in real systems because they were so slow and memory-inefficient, taking gigabytes of RAM and hard disk space to process editing traces smaller than the ones we present in this paper. And yet, the CRDTs we benchmark here, like Yjs and Automerge, can process large documents with very reasonable computational resources.

These improvements have come through a mixture of improved data structures and algorithms, and improved implementation techniques such as reduced memory allocation and better memory layout. Unfortunately, the techniques that these implementations use to achieve their performance are not well documented in either the academic literature or the documentation of those projects. It is difficult to determine which combination of factors is responsible for a performance improvement.

As a result, it is difficult to fairly compare algorithms that are implemented by different developers and use different implementation techniques. We have attempted to address this issue by writing our own reference CRDT and OT implementations that use a similar implementation style to our optimised Eg-walker implementation, enabling as much as possible an apples-for-apples comparison. We hope that future research will be able to develop more rigorous approaches to evaluating collaborative editing systems.